



Guía Completa: Despliegue VPS con Docker

Django Ecodisseny en Contabo VPS con Docker, Nginx y SSL

📖 Índice

🎯 FASE 1: PREPARACIÓN VPS

- [1.1 Especificaciones VPS Contabo](#)
- [1.2 Configuración Inicial del Sistema](#)
- [1.3 Instalación Docker y Dependencias](#)

💎 FASE 2: DESPLIEGUE DOCKER

- [2.1 Clonación del Repositorio](#)
- [2.2 Configuración con Docker Compose](#)
- [2.3 Despliegue Automatizado](#)

🌐 FASE 3: NGINX Y SSL

- [3.1 Configuración DNS Cloudflare](#)
- [3.2 Configuración Nginx como Proxy](#)
- [3.3 Certificados SSL con Certbot](#)

📊 FASE 4: MONITOREO Y MANTENIMIENTO

- [4.1 Scripts de Monitoreo Docker](#)
- [4.2 Backups Automatizados](#)
- [4.3 Troubleshooting](#)

💎 FASE 5: DESARROLLO REMOTO (OPCIONAL)

- [5.1 VSCode Remote SSH](#)
- [5.2 Configuración de Desarrollo](#)

🎯 FASE 1: PREPARACIÓN VPS

1.1 Especificaciones VPS Contabo

📊 VPS Contabo Cloud VPS 10

📱 Especificaciones Utilizadas:

- CPU: 3 vCPU Cores
- RAM: 8 GB
- Storage: 75 GB NVMe
- Región: European Union
- Precio: €3.60/mes (€43.20/año)

- └─ Ancho de banda: 32TB

🎯 Capacidad vs Uso Real:

- └─ Django Container: ~200MB RAM
- └─ PostgreSQL Container: ~150MB RAM
- └─ Nginx + Sistema: ~300MB RAM
- └─ Total utilizado: ~650MB de 8GB (muy holgado)

1.2 Configuración Inicial del Sistema

Primer Acceso y Setup Básico

```
# Conectar por primera vez (Contabo enviará credenciales por email)
ssh root@TU_IP_VPS

# Actualizar sistema
apt update && apt upgrade -y

# Configurar timezone
timedatectl set-timezone Europe/Madrid

# Configurar firewall básico
ufw allow 22      # SSH
ufw allow 80      # HTTP
ufw allow 443     # HTTPS
ufw --force enable

# Verificar configuración
ufw status
```

⚠ Configuración SSH (Opcional - Solo usuarios avanzados)

```
# OPCIONAL: Cambiar puerto SSH por seguridad
# ⚠ ADVERTENCIA: Solo hacer si tienes experiencia
# Si pierdes conexión, necesitarás usar consola VNC de Contabo

# Backup de configuración SSH
cp /etc/ssh/sshd_config /etc/ssh/sshd_config.backup

# Cambiar puerto (descomenta y cambia si quieres)
# nano /etc/ssh/sshd_config
# Buscar: #Port 22
# Cambiar a: Port 2222

# Si cambias el puerto SSH:
# systemctl restart ssh
# ufw allow 2222
# ufw delete allow 22
```

```
# ⚠ NO cierres la sesión actual hasta verificar que funciona
```

1.3 Instalación Docker y Dependencias

Instalar Docker y Herramientas Esenciales

```
# Instalar Docker oficial
curl -fsSL https://get.docker.com -o get-docker.sh
sh get-docker.sh

# Instalar Docker Compose
curl -L "https://github.com/docker/compose/releases/latest/download/docker-
compose-$(uname -s)-$(uname -m)" -o /usr/local/bin/docker-compose
chmod +x /usr/local/bin/docker-compose

# Instalar herramientas adicionales
apt install -y nginx certbot python3-certbot-nginx htop curl wget git unzip

# Verificar instalaciones
docker --version
docker-compose --version
nginx -v
certbot --version

# Habilitar Docker para iniciar automáticamente
systemctl enable docker
systemctl start docker
```



FASE 2: DESPLIEGUE DOCKER

2.1 Clonación del Repositorio

```
# Crear directorio de trabajo
mkdir -p /root/proyectos
cd /root/proyectos

# Clonar repositorio
git clone https://github.com/Mulastone/ecodisseny_dj_pg.git
cd ecodisseny_dj_pg

# Verificar branch
git branch
git status
```

2.2 Configuración con Docker Compose

Verificar Configuración Docker

El proyecto ya incluye todo lo necesario:

```
# Verificar archivos clave
ls -la docker-compose.yml      # ✔ Configuración de contenedores
ls -la Dockerfile              # ✔ Imagen Django personalizada
ls -la nginx/default.conf      # ✔ Configuración Nginx
ls -la requirements.txt         # ✔ Dependencias Python

# Verificar estructura del proyecto
tree -L 2
```

Configurar Variables de Entorno (Si es necesario)

```
# El proyecto puede funcionar sin .env personalizado
# Pero puedes crear uno si necesitas configuraciones específicas
cat > .env << EOF
DEBUG=False
SECRET_KEY=django-insecure-production-key-change-this
POSTGRES_DB=ecodisseny_db
POSTGRES_USER=ecodisseny
POSTGRES_PASSWORD=ecodisseny123
ALLOWED_HOSTS=app.arasmu.net,161.97.147.142
CSRF_TRUSTED_ORIGINS=https://app.arasmu.net
EOF
```

2.3 Despliegue Automatizado

Construcción y Inicio de Contenedores

```
# Construcción completa desde cero
docker-compose build --no-cache

# Iniciar servicios en segundo plano
docker-compose up -d

# Verificar que los contenedores están funcionando
docker-compose ps

# Debería mostrar algo como:
#
```

	Name	State	Ports
#	ecodisseny_dj_pg_db_1	Up	0.0.0.0:5433->5432/tcp
#	ecodisseny_dj_pg_web_1	Up	0.0.0.0:8000->8000/tcp

Configuración Inicial de la Aplicación

```
# Ejecutar migraciones
docker-compose exec web python manage.py migrate

# Cargar datos iniciales (fixtures)
docker-compose exec web python manage.py loaddata maestros/fixtures/*.json

# Crear superusuario (opcional, ya hay usuarios predefinidos)
docker-compose exec web python manage.py createsuperuser

# Cargar documentación
docker-compose exec web python manage.py cargar_documentacion

# Recolectar archivos estáticos
docker-compose exec web python manage.py collectstatic --noinput

# Verificar que la aplicación responde
curl http://localhost:8000
# Debe devolver HTML de la aplicación
```

🌐 FASE 3: NGINX Y SSL

3.1 Configuración DNS Cloudflare

Configurar Registros DNS

```
# En Cloudflare Dashboard > DNS > Records, añadir:

📌 Registro A para la aplicación:
  Tipo: A
  Nombre: app
  Contenido: 161.97.147.142 (tu IP VPS)
  Proxy status: DNS only (🔴 gris) - IMPORTANTE AL PRINCIPIO

📌 Registro A para admin (opcional):
  Tipo: A
  Nombre: admin
  Contenido: 161.97.147.142 (tu IP VPS)
  Proxy status: DNS only (🔴 gris)
```

Verificar Propagación DNS

```
# Verificar desde el VPS que DNS funciona
dig app.arasmu.net
nslookup app.arasmu.net

# Ambos deben devolver tu IP: 161.97.147.142
```

```
# Verificar desde otra máquina también
ping app.arasmu.net
```

3.2 Configuración Nginx como Proxy

Crear Configuración Nginx para la Aplicación

```
# Crear configuración específica para app.arasmu.net
cat > /etc/nginx/sites-available/app.arasmu.net << 'EOF'
# Configuración inicial HTTP (antes de SSL)
server {
    listen 80;
    server_name app.arasmu.net;

    # Ubicación de logs
    access_log /var/log/nginx/app.arasmu.net.access.log;
    error_log /var/log/nginx/app.arasmu.net.error.log;

    # Tamaño máximo de archivo
    client_max_body_size 50M;

    # Headers de seguridad básicos
    add_header X-Frame-Options DENY always;
    add_header X-Content-Type-Options nosniff always;
    add_header X-XSS-Protection "1; mode=block" always;

    # Servir archivos estáticos directamente desde volumen Docker
    location /static/ {
        alias /var/lib/docker/volumes/ecodisseny_dj_pg_static_files/_data/;
        expires 1y;
        add_header Cache-Control "public, immutable";
    }

    location /media/ {
        alias /var/lib/docker/volumes/ecodisseny_dj_pg_media_files/_data/;
        expires 30d;
        add_header Cache-Control "public";
    }

    # Proxy a la aplicación Django en Docker
    location / {
        proxy_pass http://127.0.0.1:8000;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
        proxy_redirect off;

        # Timeouts
        proxy_connect_timeout 30s;
    }
}
```

```
        proxy_send_timeout      30s;
        proxy_read_timeout      30s;
    }
}
EOF

# Habilitar el sitio
ln -sf /etc/nginx/sites-available/app.arasmu.net /etc/nginx/sites-enabled/

# Eliminar configuración por defecto
rm -f /etc/nginx/sites-enabled/default

# Verificar configuración
nginx -t

# Si todo está bien, reiniciar Nginx
systemctl restart nginx

# Verificar estado
systemctl status nginx
```

Verificar Nginx Funcionando

```
# Probar acceso HTTP
curl -I http://app.arasmu.net

# Debería devolver algo como:
# HTTP/1.1 200 OK
# Server: nginx/1.24.0

# Verificar logs en tiempo real
tail -f /var/log/nginx/app.arasmu.net.access.log
```

3.3 Certificados SSL con Certbot

Generar Certificados Let's Encrypt

```
# Generar certificado SSL automáticamente
# Certbot modificará la configuración de Nginx automáticamente
certbot --nginx -d app.arasmu.net --non-interactive --agree-tos --email
mulastone@hotmail.com

# Si tienes múltiples dominios:
# certbot --nginx -d app.arasmu.net -d admin.arasmu.net --non-interactive -
-agree-tos --email mulastone@hotmail.com

# Verificar certificados generados
ls -la /etc/letsencrypt/live/app.arasmu.net/
```

```
# Debería mostrar:  
# cert.pem -> ../../archive/app.arasmu.net/cert1.pem  
# chain.pem -> ../../archive/app.arasmu.net/chain1.pem  
# fullchain.pem -> ../../archive/app.arasmu.net/fullchain1.pem  
# privkey.pem -> ../../archive/app.arasmu.net/privkey1.pem
```

Verificar Configuración SSL Automática

```
# Certbot habrá modificado automáticamente /etc/nginx/sites-  
available/app.arasmu.net  
# Verificar la nueva configuración  
cat /etc/nginx/sites-available/app.arasmu.net  
  
# Debería incluir ahora:  
# - Redirección HTTP -> HTTPS  
# - Configuración SSL  
# - Headers de seguridad adicionales  
  
# Verificar configuración  
nginx -t  
  
# Reiniciar Nginx si es necesario  
systemctl reload nginx
```

Configurar Renovación Automática

```
# Verificar que la renovación automática está configurada  
systemctl status certbot.timer  
  
# Si no está activo, habilitarlo  
systemctl enable certbot.timer  
systemctl start certbot.timer  
  
# Probar renovación (dry run)  
certbot renew --dry-run  
  
# Debería mostrar: "Congratulations, all renewals succeeded"
```

Verificación Final de HTTPS

```
# Probar HTTPS  
curl -I https://app.arasmu.net  
  
# Debería devolver:  
# HTTP/2 200
```



```
# server: nginx/1.24.0

# Verificar redirección HTTP -> HTTPS
curl -I http://app.arasmu.net

# Debería devolver:
# HTTP/1.1 301 Moved Permanently
# Location: https://app.arasmu.net/

# Verificar certificado SSL
openssl s_client -connect app.arasmu.net:443 -servername app.arasmu.net < /dev/null

# Probar desde navegador
echo "👉 Acceder a: https://app.arasmu.net"
```

Optimizar Cloudflare (Opcional)

```
# Una vez que HTTPS funciona perfectamente:
# 1. Ir a Cloudflare Dashboard
# 2. Cambiar "DNS only" a "Proxied" (● naranja)
# 3. SSL/TLS mode: "Full (strict)"
# 4. Habilitar optimizaciones:
#   - Auto Minify: CSS, JS, HTML
#   - Brotli compression: ON
#   - Always Use HTTPS: ON
```

📊 FASE 4: MONITOREO Y MANTENIMIENTO

4.1 Scripts de Monitoreo Docker

Script de Monitoreo General

```
# Crear script de monitoreo específico para Docker
cat > /root/ecodisseny_dj_pg/scripts/monitor_docker.sh << 'EOF'
#!/bin/bash
echo "=== 📊 ESTADO DEL SERVIDOR $(date) ==="
echo "===== "

echo "📊 RECURSOS DEL SISTEMA:"
echo "CPU: $(top -bn1 | grep "Cpu(s)" | awk '{print $2}' | cut -d'%' -f1)% usado"
echo "RAM: $(free -h | awk 'NR==2{printf "%.1f/%.1fGB (%.1f%%)\n", $3/1024/1024,$2/1024/1024,$3*100/$2}')"
echo "Disco: $(df -h / | tail -1 | awk '{print $3 "/" $2 " (" $5 ")"}')"
```

```

echo "🐳 CONTENEDORES DOCKER:"
cd /root/ecodisseny_dj_pg
docker-compose ps

echo ""
echo "📊 USO DE RECURSOS DOCKER:"
docker stats --no-stream --format "table
{{.Container}}\t{{.CPUPerc}}\t{{.MemUsage}}"

echo ""
echo "🔧 SERVICIOS DEL SISTEMA:"
for service in nginx docker; do
    if systemctl is-active $service >/dev/null; then
        echo "✅ $service: ACTIVO"
    else
        echo "❌ $service: INACTIVO"
    fi
done

echo ""
echo "🌐 CONECTIVIDAD:"
echo "Puerto 8000: $(curl -s -o /dev/null -w "%{http_code}"
http://localhost:8000 2>/dev/null || echo 'Error')"
echo "HTTPS: $(curl -s -o /dev/null -w "%{http_code}"
https://app.arasmu.net 2>/dev/null || echo 'Error/Sin SSL')"

echo ""
echo "📁 VOLÚMENES DOCKER:"
docker volume ls | grep ecodisseny

echo ""
echo "🔍 LOGS RECIENTES (últimas 5 líneas):"
echo "--- Django ---"
docker-compose logs --tail=5 web 2>/dev/null || echo "Sin logs de Django"
echo "--- PostgreSQL ---"
docker-compose logs --tail=5 db 2>/dev/null || echo "Sin logs de
PostgreSQL"

echo ""
echo "💾 ESPACIO EN DOCKER:"
docker system df
EOF

chmod +x /root/ecodisseny_dj_pg/scripts/monitor_docker.sh

# Crear directorio scripts si no existe
mkdir -p /root/ecodisseny_dj_pg/scripts

# Ejecutar monitoreo
/root/ecodisseny_dj_pg/scripts/monitor_docker.sh

```

```
# Añadir alias útiles al bashrc
echo '# Aliases para Docker Django' >> ~/.bashrc
echo 'alias dclogs="cd /root/ecodisseny_dj_pg && docker-compose logs -f"' >> ~/.bashrc
echo 'alias dcstatus="cd /root/ecodisseny_dj_pg && docker-compose ps"' >> ~/.bashrc
echo 'alias dcrestart="cd /root/ecodisseny_dj_pg && docker-compose restart"' >> ~/.bashrc
echo 'alias monitor="cd /root/ecodisseny_dj_pg && ./scripts/monitor_docker.sh"' >> ~/.bashrc
echo 'alias cdproject="cd /root/ecodisseny_dj_pg"' >> ~/.bashrc

# Recargar bashrc
source ~/.bashrc

# Ahora puedes usar:
# monitor          # Ver estado completo
# dcstatus         # Ver contenedores
# dclogs           # Ver logs en tiempo real
# dcrestart        # Reiniciar contenedores
```

4.2 Backups Automatizados

Script de Backup Completo para Docker

```
# Script de backup específico para Docker
cat > /root/ecodisseny_dj_pg/scripts/backup_docker.sh << 'EOF'
#!/bin/bash
DATE=$(date +%Y%m%d_%H%M%S)
BACKUP_DIR="/root/ecodisseny_dj_pg/backups"
PROJECT_DIR="/root/ecodisseny_dj_pg"

# Crear directorio si no existe
mkdir -p $BACKUP_DIR

echo "📁 Iniciando backup Docker ($DATE)..."

cd $PROJECT_DIR

# 1. Backup de base de datos desde el contenedor
echo "🗄️ Backup de base de datos..."
docker-compose exec -T db pg_dump -U ecodisseny ecodisseny_db | gzip > $BACKUP_DIR/database_$DATE.sql.gz

# 2. Backup de volúmenes Docker (static y media files)
echo "📁 Backup de archivos estáticos..."
docker run --rm \
    -v ecodisseny_dj_pg_static_files:/data \
    -v $BACKUP_DIR:/backup \
    alpine tar -czf /backup/static_files_$DATE.tar.gz -C /data .
```

```

echo "📁 Backup de archivos media..."
docker run --rm \
  -v ecodisseny_dj_pg_media_files:/data \
  -v $BACKUP_DIR:/backup \
  alpine tar -czf /backup/media_files_$(date +%Y%m%d).tar.gz -C /data .

# 3. Backup del código fuente y configuración
echo "📦 Backup del código fuente..."
tar --exclude='__pycache__' \
  --exclude='.git' \
  --exclude='venv*' \
  --exclude='backups' \
  --exclude='*.pyc' \
  -czf $BACKUP_DIR/codigo_$(date +%Y%m%d).tar.gz \
  -C /root ecodisseny_dj_pg

# 4. Backup de configuración Nginx
echo "⚙️ Backup configuración Nginx..."
if [ -f /etc/nginx/sites-available/app.arasmu.net ]; then
  cp /etc/nginx/sites-available/app.arasmu.net
  $BACKUP_DIR/nginx_config_$(date +%Y%m%d).conf
fi

# 5. Limpiar backups antiguos (conservar 7 días)
echo "🧹 Limpiando backups antiguos..."
find $BACKUP_DIR -name "*_*.tar.gz" -mtime +7 -delete
find $BACKUP_DIR -name "*_*.sql.gz" -mtime +7 -delete
find $BACKUP_DIR -name "*_*.conf" -mtime +7 -delete

echo "✅ Backup completado: $(date +%Y%m%d)"
echo "📁 Archivos generados:"
ls -lh $BACKUP_DIR/*$(date +%Y%m%d)*

# Mostrar resumen de espacio
echo "📊 Resumen de backups:"
du -sh $BACKUP_DIR
df -h $BACKUP_DIR
EOF

chmod +x /root/ecodisseny_dj_pg/scripts/backup_docker.sh

# Configurar cron para backup diario a las 3 AM
(crontab -l 2>/dev/null; echo "0 3 * * *
/root/ecodisseny_dj_pg/scripts/backup_docker.sh") | crontab -

# Verificar cron
crontab -l

```

Script de Restauración

```

# Script para restaurar backups
cat > /root/ecodisseny_dj_pg/scripts/restore_backup.sh << 'EOF'
#!/bin/bash
BACKUP_DIR="/root/ecodisseny_dj_pg/backups"
PROJECT_DIR="/root/ecodisseny_dj_pg"

echo "🔄 SCRIPT DE RESTAURACIÓN"
echo "===== "

if [ -z "$1" ]; then
    echo "Uso: $0 FECHA (formato: YYYYMMDD_HHMMSS)"
    echo "Backups disponibles:"
    ls -la $BACKUP_DIR/*database* 2>/dev/null | awk '{print $9}' | sed
    's/.*database_//' | sed 's/.sql.gz//'
    exit 1
fi

DATE=$1

echo "⚠️ ADVERTENCIA: Esto restaurará el backup del $DATE"
echo "Esto SOBRESCRIBIRÁ los datos actuales."
read -p "¿Continuar? (y/N): " confirm

if [[ $confirm != [yY] ]]; then
    echo "Cancelado."
    exit 0
fi

cd $PROJECT_DIR

# Parar contenedores
echo "🛑 Parando contenedores..."
docker-compose down

# Restaurar base de datos
if [ -f "$BACKUP_DIR/database_$DATE.sql.gz" ]; then
    echo "📁 Restaurando base de datos..."
    docker-compose up -d db
    sleep 10
    gunzip < $BACKUP_DIR/database_$DATE.sql.gz | docker-compose exec -T db
    psql -U ecodisseny ecodisseny_db
else
    echo "❌ No se encontró backup de base de datos para $DATE"
fi

# Restaurar archivos estáticos
if [ -f "$BACKUP_DIR/static_files_$DATE.tar.gz" ]; then
    echo "📁 Restaurando archivos estáticos..."
    docker run --rm \
        -v ecodisseny_dj_pg_static_files:/data \
        -v $BACKUP_DIR:/backup \
        alpine sh -c "cd /data && tar -xzf
        /backup/static_files_$DATE.tar.gz"

```

```

fi

# Restaurar archivos media
if [ -f "$BACKUP_DIR/media_files_$DATE.tar.gz" ]; then
    echo "📦 Restaurando archivos media..."
    docker run --rm \
        -v ecodisseny_dj_pg_media_files:/data \
        -v $BACKUP_DIR:/backup \
        alpine sh -c "cd /data && tar -xzf
/backup/media_files_$DATE.tar.gz"
fi

# Iniciar todos los contenedores
echo "🚀 Iniciando aplicación..."
docker-compose up -d

echo "✅ Restauración completada"
EOF

chmod +x /root/ecodisseny_dj_pg/scripts/restore_backup.sh

```

4.3 Troubleshooting

Comandos de Diagnóstico Rápido

```

# Script de diagnóstico completo
cat > /root/ecodisseny_dj_pg/scripts/diagnostico.sh << 'EOF'
#!/bin/bash
echo "🔍 DIAGNÓSTICO COMPLETO DEL SISTEMA"
echo "===== "

cd /root/ecodisseny_dj_pg

echo "📄 INFORMACIÓN DEL SISTEMA:"
echo "OS: $(lsb_release -d | cut -f2)"
echo "Kernel: $(uname -r)"
echo "Uptime: $(uptime -p)"

echo ""
echo "📊 RECURSOS:"
echo "CPU: $(nproc) cores - $(top -bn1 | grep "Cpu(s)" | awk '{print $2}' |
cut -d'%' -f1)% usado"
echo "RAM: $(free -h | awk 'NR==2{printf "%.1f/%.1fGB (%.1f%%)\n",
$3/1024/1024,$2/1024/1024,$3*100/$2}')"
echo "Disco: $(df -h / | tail -1 | awk '{print $3 "/" $2 " (" $5 ")"}')'"

echo ""
echo "🐳 DOCKER:"
echo "Docker: $(docker --version)"
echo "Docker Compose: $(docker-compose --version)"
echo ""

```

```
echo "📁 Estado de contenedores:"
docker-compose ps

echo ""
echo "🔧 SERVICIOS DEL SISTEMA:"
for service in docker nginx ssh; do
    status=$(systemctl is-active $service 2>/dev/null)
    if [ "$status" = "active" ]; then
        echo "✅ $service: ACTIVO"
    else
        echo "❌ $service: $status"
    fi
done

echo ""
echo "🌐 CONECTIVIDAD:"
echo "Localhost:8000: $(curl -s -o /dev/null -w "%{http_code}"
http://localhost:8000 2>/dev/null || echo 'Error')"
if command -v dig >/dev/null; then
    echo "DNS app.arasmu.net: $(dig +short app.arasmu.net)"
fi
echo "HTTPS app.arasmu.net: $(curl -s -o /dev/null -w "%{http_code}"
https://app.arasmu.net 2>/dev/null || echo 'Error')"

echo ""
echo "📁 VOLÚMENES DOCKER:"
docker volume ls | grep ecodisseny | awk '{print " " "$2}'

echo ""
echo "🔍 PROCESOS DOCKER:"
docker ps --format "table {{.Names}}\t{{.Status}}\t{{.Ports}}"

echo ""
echo "📦 ESPACIO DOCKER:"
docker system df

echo ""
echo "📄 ÚLTIMOS LOGS (5 líneas):"
echo "--- Django ---"
docker-compose logs --tail=5 web 2>/dev/null | head -5
echo "--- PostgreSQL ---"
docker-compose logs --tail=5 db 2>/dev/null | head -5

echo ""
echo "🔥 FIREWALL:"
ufw status 2>/dev/null | head -10

echo ""
echo "🔒 SSL CERTIFICADOS:"
if [ -d "/etc/letsencrypt/live/app.arasmu.net" ]; then
    echo "✅ Certificado SSL existe"
    echo "Expira: $(openssl x509 -enddate -noout -in
/etc/letsencrypt/live/app.arasmu.net/cert.pem 2>/dev/null | cut -d= -f2)"
else
```

```
    echo "✗ No se encontró certificado SSL"
fi

echo ""
echo "=====
echo "✓ Diagnóstico completado"
EOF

chmod +x /root/ecodisseny_dj_pg/scripts/diagnostico.sh
```

Problemas Comunes y Soluciones

```
# 1. Error 502 Bad Gateway
echo "🔧 SOLUCIÓN PARA ERROR 502:"
echo "1. Verificar contenedores: docker-compose ps"
echo "2. Ver logs Django: docker-compose logs web"
echo "3. Reiniciar contenedores: docker-compose restart web"
echo "4. Verificar Nginx: nginx -t && systemctl reload nginx"

# 2. Contenedores no inician
echo "🔧 SOLUCIÓN CONTENEDORES NO INICIAN:"
echo "1. Ver logs: docker-compose logs"
echo "2. Limpiar y rebuild: docker-compose down && docker-compose build --no-cache && docker-compose up -d"
echo "3. Verificar puertos: netstat -tlnp | grep -E '(8000|5432)'"

# 3. Error de base de datos
echo "🔧 SOLUCIÓN ERROR BASE DE DATOS:"
echo "1. Verificar container DB: docker-compose exec db pg_isready -U ecodisseny"
echo "2. Ver logs DB: docker-compose logs db"
echo "3. Conectar a DB: docker-compose exec db psql -U ecodisseny ecodisseny_db"

# 4. Archivos estáticos no cargan
echo "🔧 SOLUCIÓN ARCHIVOS ESTÁTICOS:"
echo "1. Recolectar: docker-compose exec web python manage.py collectstatic --noinput"
echo "2. Ver volumen: docker volume inspect ecodisseny_dj_pg_static_files"
echo "3. Verificar Nginx: ls -la /var/lib/docker/volumes/ecodisseny_dj_pg_static_files/_data/"
```

Script de Actualización

```
# Script para actualizar la aplicación
cat > /root/ecodisseny_dj_pg/scripts/actualizar.sh << 'EOF'
#!/bin/bash
cd /root/ecodisseny_dj_pg
```



```
echo "🔄 INICIANDO ACTUALIZACIÓN"
echo "===== "

# 1. Backup de seguridad antes de actualizar
echo "📦 Creando backup de seguridad..."
./scripts/backup_docker.sh

# 2. Actualizar código desde Git
echo "📥 Actualizando código desde Git..."
git pull origin docker

# 3. Reconstruir y reiniciar contenedores
echo "🔧 Reconstruyendo contenedores..."
docker-compose down
docker-compose build --no-cache
docker-compose up -d

# 4. Esperar que arranquen
echo "🕒 Esperando que arranquen los servicios..."
sleep 15

# 5. Ejecutar migraciones si hay
echo "🔄 Aplicando migraciones..."
docker-compose exec web python manage.py migrate

# 6. Recolectar archivos estáticos
echo "📁 Recolectando archivos estáticos..."
docker-compose exec web python manage.py collectstatic --noinput

# 7. Recargar documentación
echo "📖 Recargando documentación..."
docker-compose exec web python manage.py cargar_documentacion --update

# 8. Verificar funcionamiento
echo "✅ Verificando funcionamiento..."
sleep 5
./scripts/diagnostico.sh

echo ""
echo "🎉 ACTUALIZACIÓN COMPLETADA"
echo "Verificar: https://app.arasmu.net"
EOF

chmod +x /root/ecodisseny_dj_pg/scripts/actualizar.sh
```

🖥️ FASE 5: DESARROLLO REMOTO (OPCIONAL)

5.1 VSCode Remote SSH

Configurar SSH en tu máquina local

```
# En tu máquina local, editar ~/.ssh/config:
Host ecodisseny-vps
  HostName 161.97.147.142
  User root
  Port 22
  IdentityFile ~/.ssh/id_rsa
  ServerAliveInterval 60
  ServerAliveCountMax 3
```

Conectar VSCode al VPS

```
# 1. Instalar extensión "Remote - SSH" en VSCode
# 2. Presionar F1 y escribir "Remote-SSH: Connect to Host"
# 3. Seleccionar "ecodisseny-vps"
# 4. VSCode se conectará al VPS automáticamente

# Instalar extensiones útiles en el VPS:
# - Python
# - Docker
# - GitLens
# - Python Docstring Generator
```

5.2 Configuración de Desarrollo

Comandos útiles para desarrollo con Docker

```
# Ver logs en tiempo real
docker-compose logs -f web          # Solo Django
docker-compose logs -f db          # Solo PostgreSQL
docker-compose logs -f             # Todos los servicios

# Acceder al contenedor Django
docker-compose exec web bash

# Ejecutar comandos Django
docker-compose exec web python manage.py shell
docker-compose exec web python manage.py migrate
docker-compose exec web python manage.py cargar_documentacion --update
docker-compose exec web python manage.py collectstatic --noinput

# Reiniciar solo Django (mantener DB)
docker-compose restart web

# Rebuilding tras cambios importantes
docker-compose down
docker-compose build --no-cache web
docker-compose up -d
```

```
# Ver estadísticas de recursos
docker stats

# Limpiar sistema Docker
docker system prune -f
```

Debugging

```
# Ver información detallada de contenedores
docker-compose top
docker inspect ecodisseny_dj_pg_web_1

# Verificar variables de entorno
docker-compose exec web env | grep DJANGO

# Monitoreo en tiempo real
watch -n 2 'docker-compose ps && echo "" && docker stats --no-stream'

# Verificar conectividad de red
docker network ls
docker network inspect ecodisseny_dj_pg_default
```

🎉 ¡DESPLIEGUE COMPLETADO!

✓ Estado Final del Sistema:

🌐 Aplicación: <https://app.arasmu.net>
🔧 Admin: <https://app.arasmu.net/admin/>
📖 Documentación: <https://app.arasmu.net/documentacion/>

👥 Usuarios Disponibles:

👑 ADMINISTRADORES:

- mulastone / ecodisseny2024
- gonzalo / ecodisseny2024

👤 USUARIOS REGULARES:

- sarah / ecodisseny2024
- pilar / ecodisseny2024
- santiago / ecodisseny2024
- roger / ecodisseny2024

⚡ Comandos de Gestión Rápida:

```
# Monitoreo
monitor          # Estado completo del sistema
dcstatus         # Estado de contenedores
dclogs           # Logs en tiempo real

# Mantenimiento
./scripts/backup_docker.sh    # Backup manual
./scripts/actualizar.sh      # Actualizar aplicación
./scripts/diagnostico.sh     # Diagnóstico completo

# Docker
docker-compose restart web    # Reiniciar Django
docker-compose down && docker-compose up -d # Reinicio completo
docker-compose exec web bash  # Acceder al contenedor
```

🎯 Características del Despliegue:

- ✓ **Docker Compose** con PostgreSQL y Django optimizados
- ✓ **Nginx** como proxy reverso con configuración SSL automática
- ✓ **SSL/HTTPS** con Let's Encrypt y renovación automática
- ✓ **Backups diarios** automatizados con cron
- ✓ **Monitoreo** del sistema y contenedores
- ✓ **Scripts** de despliegue, actualización y troubleshooting
- ✓ **Volúmenes persistentes** para datos, media y archivos estáticos
- ✓ **Firewall** configurado y seguridad básica implementada

📊 Uso de Recursos (VPS 8GB RAM):

```
📊 Utilización actual:
├─ Django Container: ~200MB RAM
├─ PostgreSQL Container: ~150MB RAM
├─ Sistema + Nginx: ~300MB RAM
└─ Total usado: ~650MB de 8GB (92% libre)
```

```
🎯 Capacidad de crecimiento:
├─ Margen para más aplicaciones: ~7GB
├─ Preparado para Django Oscar Shop
└─ Escalabilidad para futuras apps
```

¡Tu aplicación Django está completamente operativa en producción con Docker! 🚀🔒🔑